

HARDWARE SHOP BILLING SYSTEM

CSE2006 - Programming in Java
Project Report

A menu-driven Java console application for inventory, billing,
credit / khata management, settlement and transaction records.

Implementation and submission report
Prepared for project evaluation

1. Introduction

Small hardware shops commonly need to keep track of stock, customer credit and daily sales at the same time. This project provides a local console-based solution for these operations. The program keeps product stock, creates itemized bills, maintains khata balances and stores transaction history in SQLite.

The design was kept close to the CSE2006 Programming in Java syllabus. It uses Java classes and constructors, encapsulation, inheritance, abstraction, polymorphism, overloading, exception handling, collections, file handling,

multithreading and JDBC.

2. Problem Statement

Manual stock and credit records can result in wrong quantities, repeated calculations and difficulty checking outstanding dues. The proposed system centralizes the basic billing workflow and validates user input before committing changes.

3. Objectives

Objective	How the project addresses it
Billing	Create walk-in and credit sales with itemized totals and receipts.
Inventory	Show current stock and prevent sales above available quantity.
Khata	Store credit customer details, current dues and settlement history.
Persistence	Use SQLite through JDBC for important application records.
Reliability	Use validation, domain exceptions and database commit / rollback.
Java concepts	Combine the course OOP, collections, I/O, thread and JDBC topics in one application.

4. Functional Requirements

FR1 - Display inventory with product ID, name, price and current stock.

FR2 - Create walk-in sales and save receipts.

FR3 - Create credit / khata sales and update customer balance.

FR4 - Validate product IDs, quantities and customer type.

FR5 - Settle customer dues and record the payment.

FR6 - View recent transaction history.

FR7 - Persist critical data in SQLite.

FR8 - Search products by name.

FR9 - Handle invalid input and database errors without crashing.

5. Non-functional Requirements

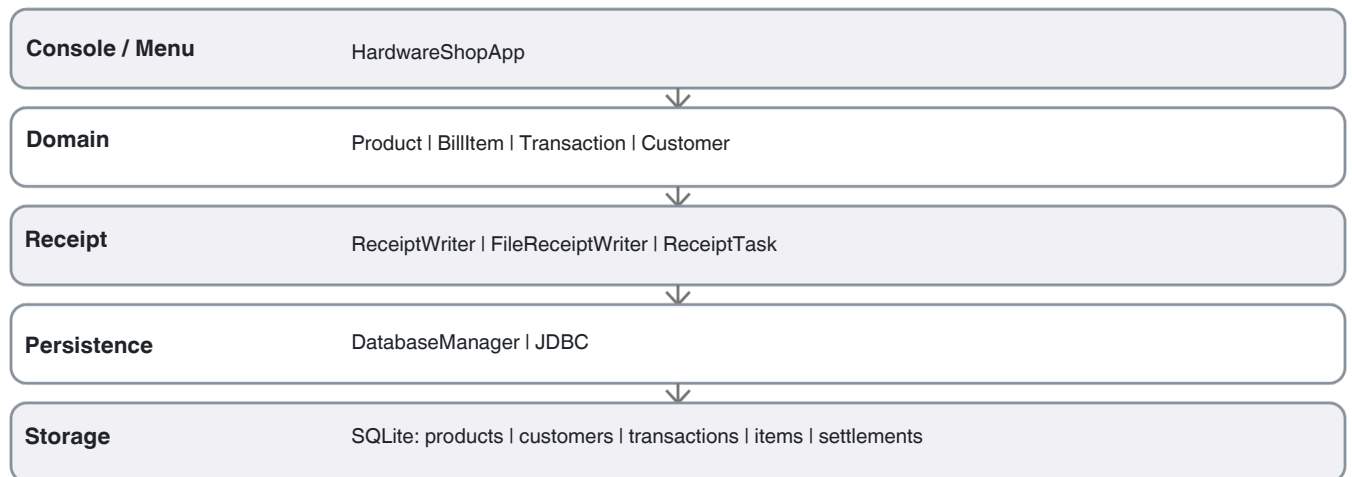
Usability: menu-driven and easy to follow. Reliability: sale changes use one database transaction. Maintainability: responsibilities are separated into small Java classes. Validation: invalid data is rejected before or during a sale.

Portability: Java 17+ with the SQLite JDBC driver.

6. System Architecture

The application follows a simple layered structure. The console layer collects input, domain classes hold the business data, receipt classes handle file output, and DatabaseManager handles JDBC persistence.

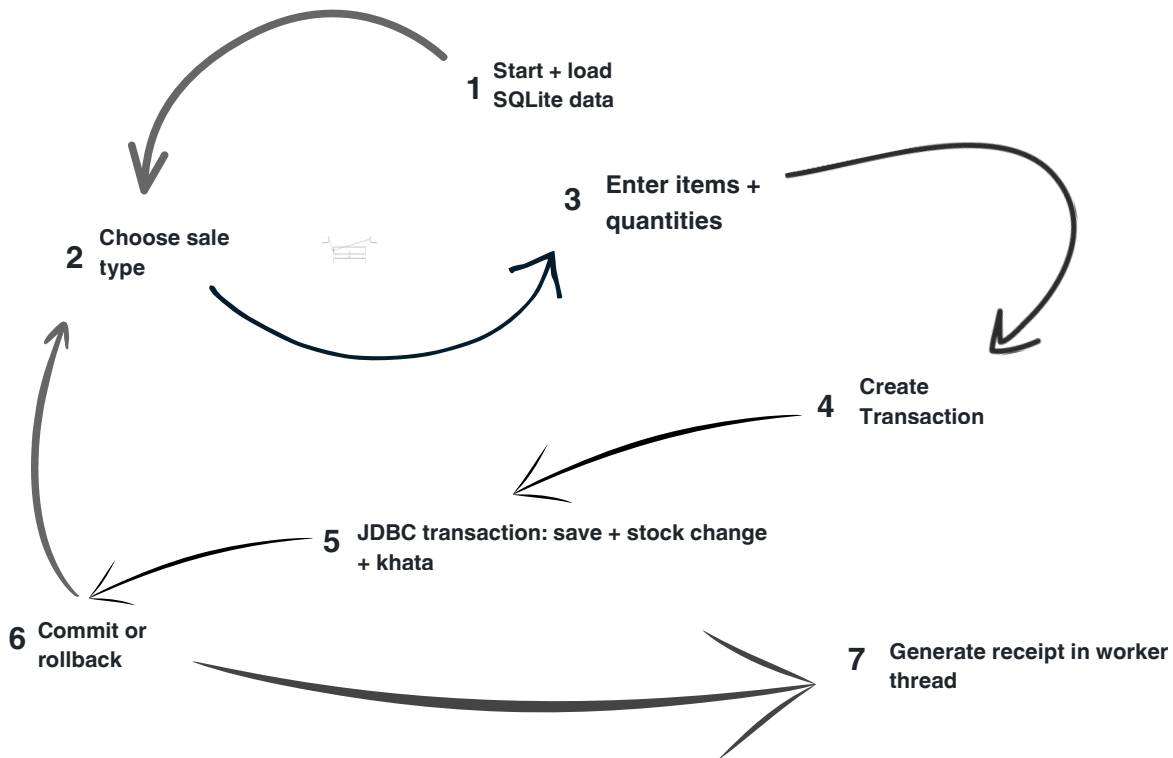
Architecture overview



Main data flow: the operator works through HardwareShopApp, the domain objects represent the bill, DatabaseManager persists the changes, and the receipt service creates the text receipt after commit.

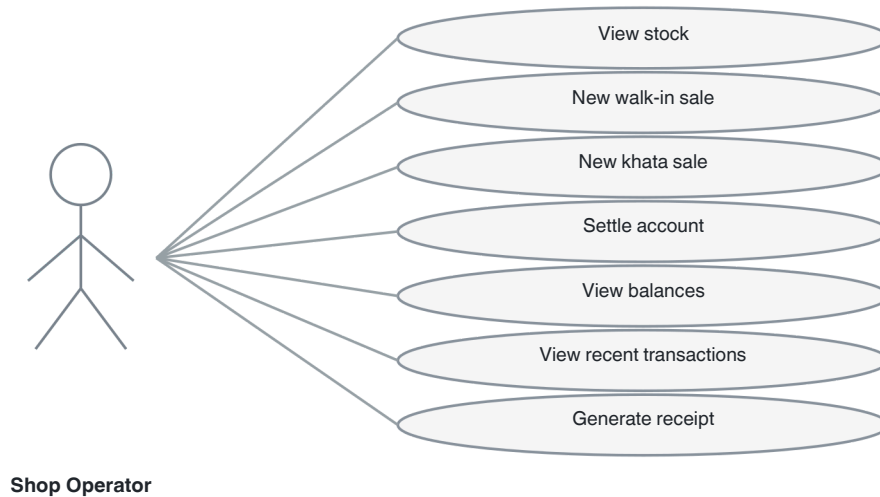
7. Process Workflow

For a sale, customer type is checked first. Items are collected without directly changing the database. After the Transaction object is created, the database layer validates stock, writes the sale records and commits all related changes together. Only after commit does the program update the loaded objects and start receipt writing.



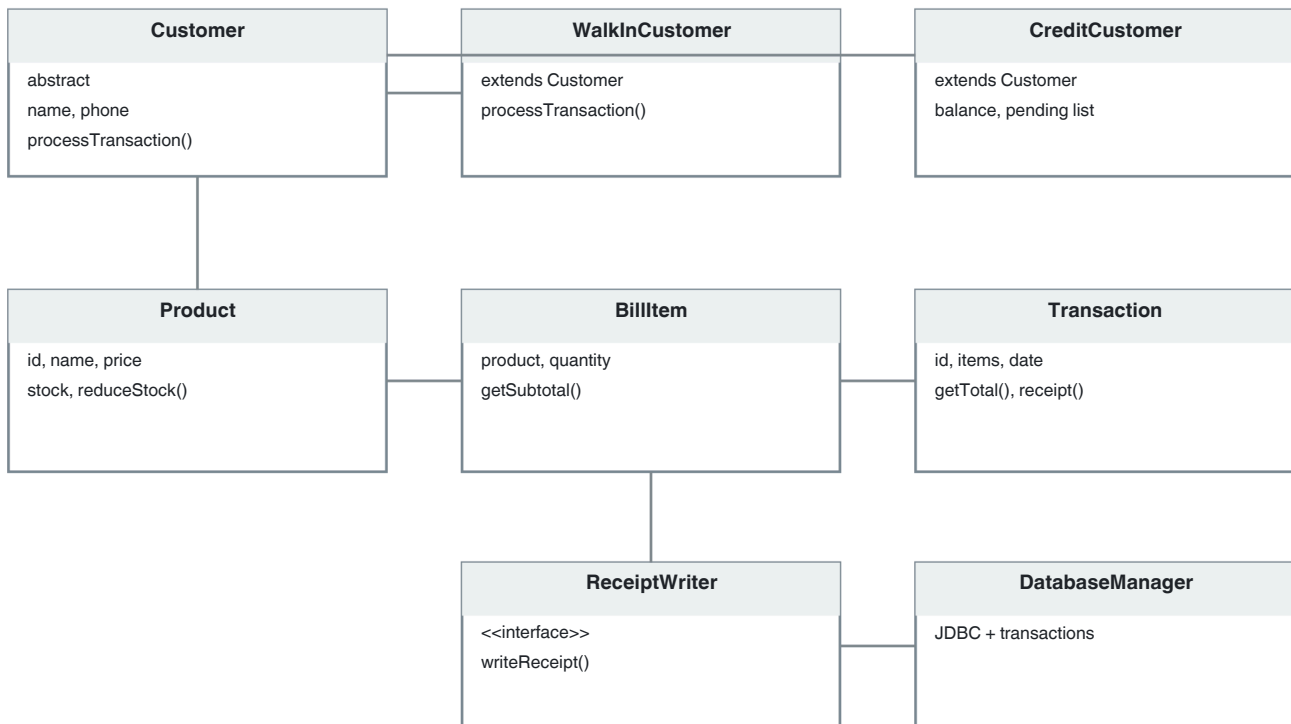
8. Use Cases

The main actor is the shop operator. The supported use cases are viewing stock, searching products, making walk-in or khata sales, generating receipts, viewing balances, settling accounts and viewing recent transactions.



9. Class Design

Customer is the common abstract class. WalkInCustomer and CreditCustomer specialize customer behavior. Product represents inventory, BillItem represents one line of a bill, and Transaction groups the bill items. ReceiptWriter is an interface so receipt output is separate from the main billing flow.



10. Design Decisions and Rationale

Concept	Use in the project
Encapsulation	State such as Product stock and Transaction items is private and changed through methods.
Abstraction	Customer is an abstract class for common customer data and behavior.
Inheritance	WalkInCustomer and CreditCustomer extend Customer.
Polymorphism	Each customer subclass provides its own processTransaction() implementation.
Overloading	Transaction has two constructors, including one useful when restoring stored records.
Exception handling	Custom checked exceptions cover product, stock and settlement errors.
Collections	Map and List structures hold products, customers and bill items.
I/O	FileReceiptWriter creates receipt text files.
Multithreading	ReceiptTask implements Runnable and performs receipt writing on a worker thread.
JDBC	PreparedStatement, ResultSet, foreign keys and commit / rollback provide persistence.

11. Database Design

SQLite storesthemainapplicationdata. A transaction header is separated from transaction_items so one sale can contain multiple products. Credit customers have a current balance, while settlements keep a historical record of payments.

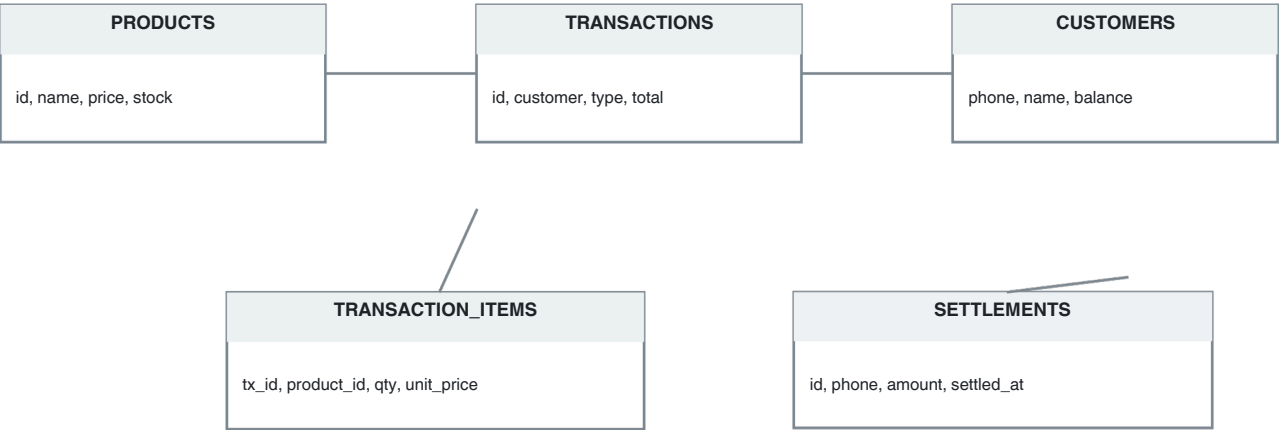


Table	Purpose
products	Product master with price and current stock.
customers	Credit customer details and current balance.
transactions	Sale header, date, customer type and total.
transaction_items	Products and quantities belonging to a sale.
settlements	Recorded payments against customer dues.

A sale is treated as one JDBC transaction. If stock validation, insertion or another SQL step fails, the connection rolls back, preventing a half-saved sale.

12. Implementation Details

At startup, DatabaseManager creates the required tables if they are missing and seeds a small product list for a new database. Products and existing credit customers are loaded into collections. Transaction IDs continue from the highest persisted transaction ID.

During a sale, the application first validates the customer type and then collects items. Duplicate entries for the same product are merged. Before the final commit, DatabaseManager checks the current database stock again. For credit sales it also creates or updates the customer balance. The receipt is generated only after a successful commit.

The program uses PreparedStatement rather than building SQL by string concatenation. Generated receipts use Java NIO file APIs, and ReceiptTask demonstrates the course multithreading topic in a small, isolated way.

13. Testing Approach

Test	Scenario	Expected / observed result
T01	Compile all Java files	Passed in build environment
T02	Stock greater than available	InsufficientStockException is caught
T03	Zero quantity	Input is rejected
T04	Transaction total	Calculated from item price x quantity
T05	Credit balance + settlement	Domain validation test passed
T06	Receipt worker	ReceiptTask test passed
T07	JDBC persistence / rollback	Covered by SystemTest; run after JDBC driver setup

Automated validation output from the build environment:

```
Added toTest Customer'skhata. Balance: Rs.20.00
All validation and business-logic tests passed.
```

Manual evidence still required: capture real screenshots from the local application run for the menu, inventory, walk-in bill, khata bill, settlement, transaction history and invalid input.

14. Challenges and Solutions

Challenge	Solution
Stock could become inconsistent after a failed sale	Move stock updates into the same JDBC transaction as the sale and use rollback.
Credit balance disappeared after restart	Persist customers and balances in SQLite.
A product could be entered twice	Merge repeated product entries before creating the Transaction.
Receipt code could mix with billing logic	Use ReceiptWriter and FileReceiptWriter as separate classes.
Invalid customertypecould be detected too late	Validate the choice before collecting any items.

15. Learnings

The project provided practice with object-oriented design, collection handling, checked exceptions, file output and JDBC. A major learning point was that a sale is not only a total calculation: several pieces of data must be changed consistently. Using commit and rollback made this relationship explicit.

16. Future Enhancements

Possible future work includes a GUI, product add / edit screens, user authentication, richer reports and PDF invoices. These are deliberately outside the current local console scope.

17. Project Structure

```
src/  
  Product.java, BillItem.java, Transaction.java  
  Customer.java, WalkInCustomer.java, CreditCustomer.java  
  ReceiptWriter.java, FileReceiptWriter.java, ReceiptTask.java  
DatabaseManager.java, HardwareShopApp.java  
  ProductNotFoundException.java, InsufficientStockException.java  
NoOutstandingBalanceException.java  
  ValidationTest.java, SystemTest.java  
docs/  
  design.md, testing.md, rubric_coverage.md, submission_checklist.md, viva.md  
README.md, statement.md, schema.sql  
run.bat, test.bat, setup_driver.bat, run.sh, test.sh
```

18. References

1. Oracle Java documentation - Java language, collections, I/O and JDBC concepts.
2. SQLite documentation - SQL tables, constraints and transactions.
3. Xerial SQLite JDBC project documentation - JDBC driver usage and current dependency information.
4. CSE2006 Programming in Java course syllabus and project submission requirements supplied for this project.

19. Submission Evidence Checklist

Before uploading, run the program locally, verify the automated tests, capture real screenshots, review the GitHub folder structure and make sure generated database / build files are excluded.

12. Screenshots / Results

Real console output captured from the running project. Personal contact details have been redacted in the copies included for submission.

Successful credit/khata sale

```
===== HARDWARE SHOP BILLING SYSTEM =====
1. New sale
2. Settle khata account
3. View stock
4. View balances
5. View recent transactions
6. Search product by name
7. Exit
Choose: 1
Customer type (1-Walk-in, 2-Khata): 2

===== HARDWARE SHOP BILLING SYSTEM =====
ID  PRODUCT          PRICE    STOCK
P1  Paint (1L)        Rs.350.00 Stock: 40
P2  Paint Brush       Rs.60.00  Stock: 100
P3  Cement Bag        Rs.400.00 Stock: 25
P4  Nails (1kg)       Rs.120.00 Stock: 60
P5  Sandpaper         Rs.20.00  Stock: 200
Product ID (done to finish): p1
Quantity: 10
Item added.
Product ID (done to finish): p4
Quantity: 40
Item added.
Product ID (done to finish): done
Added to manthan's khata. Balance: Rs.8300.00
Bill saved to bills\bill_manthan_1001.txt

Transaction: 1001
Date: 18-09-2026 11:53

=====
Paint (1L)          10 x Rs.350.00 = Rs.3500.00
Nails (1kg)         40 x Rs.120.00 = Rs.4800.00
=====
TOTAL: Rs.8300.00

===== HARDWARE SHOP BILLING SYSTEM =====
1. New sale
2. Settle khata account
3. View stock
4. View balances
5. View recent transactions
6. Search product by name
7. Exit
Choose: |
```

Khata settlement

```
===== HARDWARE SHOP BILLING SYSTEM =====
1. New sale
2. Settle khata account
3. View stock
4. View balances
5. View recent transactions
6. Search product by name
7. Exit
Choose: 2

=====
ACCOUNT SETTLED FOR MANTHAN. Amount paid: Rs.8300.00
```

Recent transaction history

===== HARDWARE SHOP BILLING SYSTEM =====

1. New sale
2. Settle khata account
3. View stock
4. View balances
5. View recent transactions
6. Search product by name
7. Exit

Choose: 5

RECENT TRANSACTIONS

#1001		2026-09-18 11:53:13		manthan			CREDIT			Rs.8300.00
-------	--	---------------------	--	---------	--	--	--------	--	--	------------

Product-name search

===== HARDWARE SHOP BILLING SYSTEM =====

1. New sale
2. Settle khata account
3. View stock
4. View balances
5. View recent transactions
6. Search product by name
7. Exit

Choose: 6

Product name: paint

MATCHING PRODUCTS

P1	Paint (1L)	Rs.350.00	Stock: 30
P2	Paint Brush	Rs.60.00	Stock: 100

View stock

```
===== HARDWARE SHOP BILLING SYSTEM =====
```

1. New sale
2. Settle khata account
3. View stock
4. View balances
5. View recent transactions
6. Search product by name
7. Exit

Choose: 3

ID	PRODUCT	PRICE	STOCK
P1	Paint (1L)	Rs.350.00	Stock: 30
P2	Paint Brush	Rs.60.00	Stock: 100
P3	Cement Bag	Rs.400.00	Stock: 25
P4	Nails (1kg)	Rs.120.00	Stock: 20
P5	Sandpaper	Rs.20.00	Stock: 200

Quantity validation

```
===== HARDWARE SHOP BILLING SYSTEM =====
```

1. New sale
2. Settle khata account
3. View stock
4. View balances
5. View recent transactions
6. Search product by name
7. Exit

Choose: 1

Customer type (1-Walk-in, 2-Khata): 1

Customer name: test customer

ID	PRODUCT	PRICE	STOCK
P1	Paint (1L)	Rs.350.00	Stock: 30
P2	Paint Brush	Rs.60.00	Stock: 100
P3	Cement Bag	Rs.400.00	Stock: 25
P4	Nails (1kg)	Rs.120.00	Stock: 20
P5	Sandpaper	Rs.20.00	Stock: 200

Product ID (done to finish): p1

Quantity: 0

Quantity must be positive.

Product ID (done to finish): |

Insufficient-stock validation

===== HARDWARE SHOP BILLING SYSTEM =====

1. New sale
2. Settle khata account
3. View stock
4. View balances
5. View recent transactions
6. Search product by name
7. Exit

Choose: 1

Customer type (1-Walk-in, 2-Khata): 1

Customer name: test customer2

ID	PRODUCT	PRICE	STOCK
P1	Paint (1L)	Rs.350.00	Stock: 30
P2	Paint Brush	Rs.60.00	Stock: 100
P3	Cement Bag	Rs.400.00	Stock: 25
P4	Nails (1kg)	Rs.120.00	Stock: 20
P5	Sandpaper	Rs.20.00	Stock: 200

Product ID (done to finish): p5

Quantity: 3000

Not enough stock for Sandpaper. Available: 200

Product ID (done to finish): |